

Thinking in Python

EN.553.285 Introduction to Scientific Programming in Python | Winter 2026

Zan Ahmad · Johns Hopkins University

Edited lecture sample from January 8, 2026 (Lecture 1).

Scientific programming requires more than syntax. We need to explain what code does, choose representations that fit the problem, and anticipate how the work grows with the size of the input. These notes condense the conceptual core of the original lecture.

1. Names refer to objects

Assignment binds a name to an object. It does not automatically copy that object. In the example below, both names refer to the same mutable list:

```
a = [1, 2, 3]
b = a
b.append(4)
print(a) # [1, 2, 3, 4]
```

A function can likewise modify a mutable object passed to it. When mutation is intended, document it; when it is not, return a new result. A shallow copy of a list copies its outer container, but nested objects can still be shared.

```
def with_extra_value(values, extra):
    return values + [extra]

original = [1, 2]
updated = with_extra_value(original, 10)
assert original == [1, 2]
```

2. Choose the representation

List: use when order and indexing matter, such as a time series or simulation output. Lists may contain repeated values.

Dictionary: use for a mapping from keys to values, such as counts, labeled observations, or configuration settings. Keys must be hashable.

Set: use for uniqueness and membership, such as the labels encountered so far. Do not rely on a meaningful iteration order.

3. Recognize the pattern

Many tasks combine accumulation (a running total), transformation (unit conversion), filtering (selecting valid observations), counting (frequencies), and two-pass logic (first compute a summary, then use it). Before writing the loop, name the pattern and the information you must keep.

4. Reason about growth

Big-O notation describes an asymptotic bound on resource use as input size n increases. It is not a prediction of exact seconds. State what n means, which operation you count, and whether your claim is worst-case, average-case, or amortized.

For ordinary Python lists, indexing and `len(a)` are $O(1)$; scanning with `sum(a)`, `max(a)`, or a membership test is $O(n)$, assuming constant-cost element operations. Appending is $O(1)$ amortized. Inserting near the front can require $O(n)$ movement.

Dictionary and set lookup are $O(1)$ on average under ordinary hashing assumptions; pathological collisions can worsen that bound. A nested all-pairs comparison is $O(n^2)$, while comparison sorting is typically $O(n \log n)$. These are growth statements, not guarantees that one implementation wins on every small input.

A short worked example: frequency counts

```
def frequencies(values):
    counts = {}
    for value in values:
        counts[value] = counts.get(value, 0) + 1
    return counts

print(frequencies(["rest", "walk", "rest"]))
# {"rest": 2, "walk": 1}
```

For n hashable values and k distinct keys, this uses $O(n)$ expected time and $O(k)$ additional storage under the usual hash-table assumptions. It remembers each count instead of rescanning the entire input for every key.

Practice prompts

Explain how aliasing differs from copying. Give an example where a shallow copy still shares an inner list. Choose a data structure for time-ordered readings, unique sensor names, and sensor-name-to-calibration mappings. Finally, compare a two-loop duplicate check with a set-based approach, stating the time and space assumptions.

From Python to NumPy

These same patterns appear in array programming. Vectorization can reduce interpreter overhead and use compiled numerical routines, but it does not automatically fix a poor algorithm or eliminate memory costs. Check correctness and array shape before focusing on speed.

Source and revision. Zan Ahmad, Johns Hopkins University, Lecture 1 (January 8, 2026). Edited October 2026; classroom logistics removed, the original `len(list)` complexity error corrected, and average/amortized bounds and shallow-copy behavior clarified. The course reference was Robert Johansson's Lectures on scientific computing with Python (CC BY 3.0).